

Simulating Federated Learning using FedAvg

Mohammed Aftaab Bin Usman*

*Department of Computer Science, Illinois Institute of Technology; Chicago, IL, United States

Abstract—This assignment implements a Federated Learning (FL) simulation for image classification using non-IID data partitions across multiple simulated devices. A ResNet-18-based neural network is trained concurrently on client datasets using a ThreadPoolExecutor to emulate distributed learning. The final global model is then deployed via a Flask-based web application, containerized using Docker, to allow inference requests over the Internet. Results demonstrate the effectiveness of FL for collaborative learning while preserving data privacy.

Index Terms—Federated Learning, Non-IID Data, ResNet-18, Docker, Flask, Model Deployment, CIFAR-10

I. INTRODUCTION

Deep learning models have shown remarkable promise in the image classification domain, training such models typically requires aggregating large amounts of data at some centralized location. This raises privacy concerns in some applications, since sensitive data from a user’s device might need to be shared.

This issue can be resolved using federated learning (FL), which allows multiple devices to train a global model, without them ever needing to transfer their raw data to a central server [1]. Instead, each device trains locally on its own data and only sends model updates to the server, which then combines them.

In this assignment, a federated learning setup is simulated for image classification on the CIFAR-10 dataset. To make it more realistic, non-IID splits of data across clients are intentionally created, to simulate class imbalance across devices. After training the global model, it is deployed as a simple web-application that can perform inference on new images, portraying how FL can be applied in a practical context.

II. METHODOLOGY

A. FL Simulation

A simple FL setup was made from scratch using Pytorch, main idea was to simulate multiple nodes, each with its unique local dataset and training process, the simulation followed the standard federated averaging (FedAvg) approach, where multiple clients train local models on their own data and a central server aggregates these models to form a global model. This approach is widely used across research in FL due to its simplicity and effectiveness [2].

CIFAR-10 dataset partitioned across 64 clients in a non-IID manner is used, while ResNet18 servers as the backbone for each client’s local training. Following this, the client model’s parameters are updated to a central server, averaged to update the global model.

Each client’s accuracy and per-batch training loss were

recorded during the training procedure. After each communication round, the global model is evaluated on a separate validation set to measure its generalization performance.

B. Web Application

A simple web-application is created using Flask, as a way for the user to interact with the trained model and perform inference.

Docker is used to containerize the web-application to ensure reproducibility and to deploy on Chameleon Cloud. The modular design separates the core FL training script and web interface, making updating and redeploying components easier.

III. EXPERIMENTAL DESIGN & RESULTS

The FL simulation was run on 64 clients with non-IID partitions from the CIFAR-10 dataset. Each client trained its own instance of ResNet-18 model on its own data for two epochs every communication round. A total of 50 communication rounds were performed to allow the global model to converge. The FedAvg strategy is used to aggregate the client update data. Each client’s batch-level training loss and accuracy are recorded, and the global model is tested on a different validation set following each communication round.

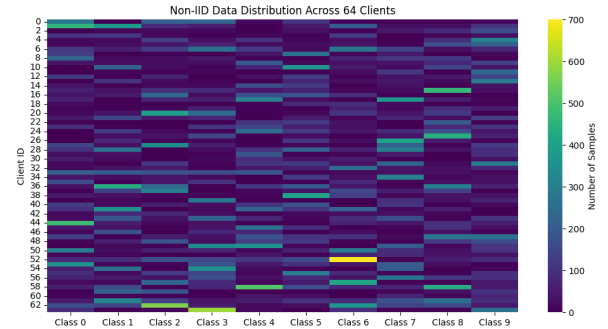


Fig. 1. Non-IID Distribution of data across 64 clients

The client data was partitioned using a Dirichlet distribution with concentration parameter $\alpha = 0.5$, which added a moderate degree of heterogeneity among clients. A lower α value (e.g., 0.1) would create a more skewed and imbalanced data distribution, increasing the difficulty of achieving global convergence since each client would specialize heavily in certain classes. In contrast, a higher α (e.g., 1.0 or greater) would make the data more uniformly distributed, which leads to smoother convergence and potentially higher validation accuracy. Furthermore, increasing the number of local epochs

or total communication rounds would allow better convergence of the global model but may also increase overfitting if local client updates dominate the global aggregation.

The main metrics tracked were:

- **Train Loss and Accuracy:** To track learning trends, the average loss and accuracy for all clients per round is calculated.
- **Validation Loss and Accuracy:** After aggregating client models, the global model was tested on a held-out validation set to determine generalization.

The training loss decreased steadily after each communication round while validation accuracy increased, indicative of effective learning across clients despite the non-IID data. Following the last communication round, the global model achieved a validation loss of **0.7111** and a validation accuracy of **76.67%**, demonstrating that the FedAvg approach can handle heterogeneous client data successfully.

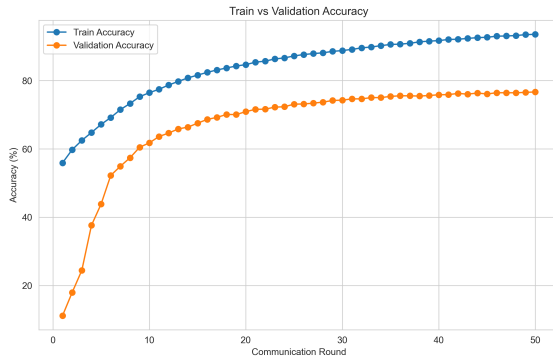


Fig. 2. Training vs Validation Accuracy across Communication Rounds

Train Accuracy rapidly increases and stabilizes near 95%, significantly outperforming the Validation Accuracy, which plateaus around 77.5%. This clear divergence indicates the model is overfitting to the training data.

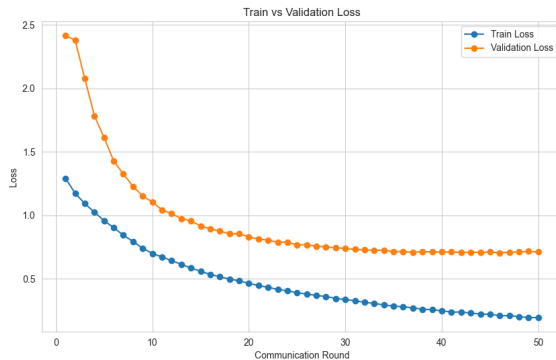


Fig. 3. Training vs Validation Loss across Communication Rounds

Both loss metrics decreased as training progressed, the Train Loss drops rapidly to nearly 0.15, reflecting successful opti-

mization on the training data. The Validation Loss stabilized a bit earlier at a higher value nearly 0.7.

```
PS C:\Users\Aftab\Desktop\fedlearn-project\code> curl.exe -X POST -F "file=@test_image.png" http://127.0.0.1:8000/predict ("prediction":5)
```

Fig. 4. Inference request via web app

I submitted a test image (dog) to the "/predict" endpoint of the web service (<http://127.0.0.1:8000>) and received a correct prediction of class label 5. In CIFAR-10, class label 5 corresponds to dog.

These results can summarize that the FL training was simulated successfully and approximated the performance of centralized training while preserving data locality and client privacy. The metrics confirm that even with heterogeneous client data, collaborative learning via FedAvg is effective.

IV. CONCLUSIONS

The goal of the assignment was to explore FL for image classification on non-IID data and demonstrate how this approach can be used to deploy models for broader access.

Motivation behind this work can be associated to the growing need of training models collaboratively across distributed data sources without compromising data privacy. My simulation, based on federated averaging approach, demonstrates that even in cases where data is not uniformly distributed among clients, a good model can be trained.

For future reference, the work can be expanded in several ways, one such direction is experimenting with more sophisticated aggregation algorithms like FedProx or FedNova, to handle client heterogeneity better.

Another promising approach is to explore privacy-preserving techniques such as differential privacy or secure aggregation to improve data confidentiality. Lastly, applying this framework to more complex datasets and carrying out larger-scale experiments on cloud platforms may shed more light on the robustness and scalability of federated learning.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *AISTATS*, 2017.
- [2] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.